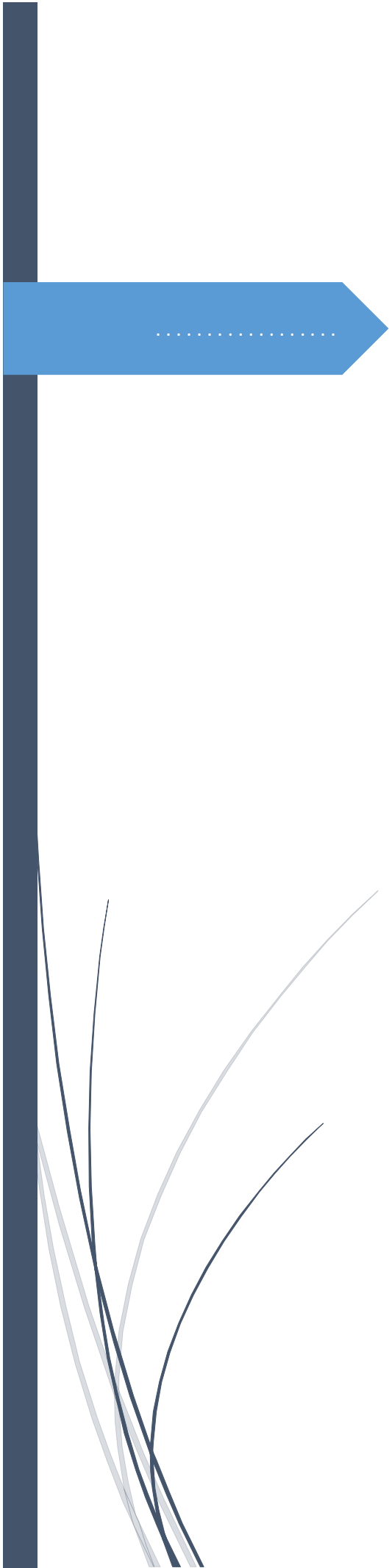




RAPPORT TECHNIQUE DU JEU

.....

SOMMAIRE :

I. ARCHITECTURE :

1. *Server*
2. *System*
3. *Resources*

II. PACKAGES/CLASSES :

1. *Buildings Package*

- 1.1 Main Classes
- 1.2 Buildings Effects Subpackage

2. *Units Package*

- 2.1 Main Classes
- 2.2 Operational and Technical Classes

3. *Economy Package*

- 3.1 Classes

4. *Factors Package*

- 4.1 Classes

5. *Interaction Package*

- 5.1 Classes

6. *Menu Package*

- 6.1 Classes

7. *Stats Package*

- 7.1 Classes

8. *UI Package*

- 8.1 Classes

9. *World Package*

- 9.1 Classes

10. *Global Package*

- 10.1 Classes

11. *Gameplay Package*

- 11.1 Classes

III. POSITIVE POINTS

IV. NEGATIVE POINTS

V. CONCLUSION

I. THE ARCHITECTURE :

We used the System/Server/Resource Architecture which is by definition :

An architecture that organizes a game (or any complex application or system) into three clear layers that work together.

1. Server :

The server is the central controller of the game. It manages the main loop, initializes the Systems and Resources (update()) and ensures that everything happens in the correct order.

2. System :

Systemes implements the game rules and logic. Each System handles a specific responsibility, such as movement, combat, economy or UI. It manipulates Resources to

apply the « Rules » of the game .

3. Resources :

Resources are the entities (or objects) of the game such as : Units, Buildings...ect . They store state and attributes (HP, Price, Name...) but contain little or no logic. They are manipulated by Systems but do not coordinate themselves .

II. THE PACKAGES/CLASSES

There is a total of 10 packages and about 68 classes. The packages are as follows :

1. Buildings Package :

The Buildings package contains classes related to building-type elements and their management.

1.1.Main Classes :

A. **Building2D** : *represents a two-dimensional building object.*

B. **BuildingEffect** : *defines effect-related data associated with buildings.*

C. **BuildingResource** : *represents resources linked to buildings.*

D. **BuildingRules** : *contains rule definitions related to buildings.*

E. **BuildingRuntimeServer** : *manages ongoing buildings created*

F. **BuildingServer** : *acts as a server-side handler for building data*

G. BuildingSystem : *coordinates building-related processing at the system level.*

1.2. Buildings Effects Subpackage :

The Buildings.Effects subpackage contains effect classes specific to buildings.

A. NoEffect represents : *a neutral or empty effect.*

B. PopulationEffect : *represents an effect related to population values.*

2. Units Package :

The Units package contains classes related to Units-type elements and their management and their fonctionement.

2.1.Main Classes :

- A. **Unit2D** : represents a 2-dimensional Unit object.
(position, movement).
- B. **UnitResource** : Stores unit attributes (HP, attack, cost).
- C. **UnitRules** : Defines rules for unit behavior (combat, movement).
- D. **UnitRuntimeServer** : Manages active units during gameplay.
- E. **UnitServer** : Handles server-side unit data.
- F. **UnitSystem** : Coordinates unit logic at system level.
- G. **UnitController** : Central controller for unit behavior, coordinating movement, combat, and interaction with other systems.

2.2.Operational and Technical Classes :

- A. **CombatSystem** : Implements *combat mechanics* for units, handles *attack resolution*, damage calculation, and *updates* unit states. It also works closely with *UnitRules* and *UnitResource*.
- B. **MovementSystem**:Manages unit movement logic, including speed, direction, collision detection, and terrain restrictions. Interfaces with *Pathfinder*.
- C. **AnimationSystem**:Controls animations for units (movement, attack, idle states). Ensures visual feedback *aligns* with unit actions.

D. **Path**: Represents a **path object** (sequence of coordinates or waypoints for unit movement)

E. **Pathfinder**: Provides **algorithms to calculate optimal paths** across the world map. Considers terrain, obstacles, and unit-specific movement rules. Outputs **Path objects** for the MovementSystem. Uses data from WorldMap and TerrainResource **to evaluate movement costs**. Interfaces with MovementSystem **to guide unit movement along calculated paths**.

3. Economy Package:

The Economy package groups classes related to economic data and processing.

3.1Classes:

- A. **EconomyResource** : *represents economy-related resources.*
- B. **EconomyRules** : *contains rules associated with economic logic.*
- C. **EconomySystem** : *manages economy-related processes.*

4. Factors Package:

The Factors package contains classes that represent influencing values or modifiers.

4.1Classes:

- A. **Factor** defines a general factor structure for modifiers (e.g., terrain factor, Fear ...)
- B. **FactorEffect** represents an effect linked to a factor (e.g., reduced movement speed).
- C. **FactorRules** contains rules related to factor evaluation

5. Interaction Package :

The Interaction package contains classes related to user or unit interaction handling.

3.1 Classes:

- A. **InteractionSystem** coordinates interaction-related processes
- B. **UnitModeController** manages different modes related to units.
- C. **UnitMode** represents the available unit modes.

D. ***UnitOrderSelector*** handles the selection
of unit orders.

E. ***UnitSelector*** manages unit selection logic.

6. Menu Package

The Menu package contains classes related to menu interfaces.

6.1Classes:

- A. **. MenuInterface** : *defines the main menu interface structure.*
- B. **StartButton** : *represents a button used to initiate actions from the menu.*

7. Stats Package

The Stats package contains classes related to statistical data and processing

7.1Classes:

- A. **StatsResource** : *represents statistical data resources.*

- B. **StatsRules** : defines rules related to statistics handling.
- C. **StatsSystem** : manages statistical operations at the system level.

8. UI Package

The UI package contains user interface related classes.

8.1Classes:

A. BuildModeControllerPannel : manages build-mode interface behavior.

B. BuildingSelectorPanel : represents a panel used to select buildings.

C. BuildingSelectorPanelButton : represents a button within the building selector panel.

D. UnitSelectorPanel:Panel for selecting units.

E.

UnitSelectorPanelButton:Button within the unit selector panel.

*F. **StatsPanel** :Displays player statistics.*

*G. **MenuPanel**:Provides navigation between game menus.*

*H. **NotificationPanel** :Shows alerts, warnings, or system messages.(didnt have time to implement it).*

*I. **AnimationPanel**:Handles UI animations and transitions.*

9. World Package :

The World package contains classes that describe the overall environment.

9.1Classes:

- A. WorldMap** :represents the world map structure.
- B. WorldRules**:contains rules related to the world.
- C. WorldSystem** :manages world-level processing.
- D. WorldResource** :Manages resources within the world.
- E. TerrainResource**:Defines terrain types and properties.
- F. PlacementGridResource**:Handles grid-based placement of objects.

G. **WorldServer** :*Server-side
synchronization of the world
state.*

10.Global Package :

10.1Classes:

- A. **GlobalConfig**: Stores global configuration values (constants, settings..).
- B. **GlobalEventHandler** :Manages global events across systems.
- C. **GlobalRules** : Defines overarching rules that apply to multiple systems.
- D. **GlobalResource** :Shared resources accessible across packages.

11.Gameplay Package :

11.1Classes:

- A. **GameplaySystem** : Core system implementing game rules and progression.
- B. **GameplayRules** :Defines rules for missions, win/lose conditions, and progression.

*C. **GameplayResource** : Stores
gameplay-related data
(objectives, states ...)*

III. POSITIVE POINTS :

-  **Good modular design:** Each subsystem (economy, buildings, world, UI) is separated.
-  **Scalability:** The architecture allows expansion (new buildings, units, rules).
-  **Clear naming conventions:** Files are named according to their responsibility.

IV. NEGATIVE POINTS :

-  Many resource classes like *TerrainResource* and

WorldResource may overlap in responsibility.

- + Maintainability risk*
- + Grid-based systems and server logic can be heavy if not optimized. (So a lack of Optimization)*

V. CONCLUSION :

Each class was presented based on its name and placement within the package structure, without interpreting internal logic or intended behavior. The package organization reflects a clear separation of responsibilities and integrates with the NeutronStar Game Engine at the system level. Eventhough there are a few imperfections to this game, it is

overall well structured and nicely organized.